

EmotionBudgetApp 프로젝트 구조 및 핵심 코드 설명

Android 프로그래밍 기말 프로젝트 제출용 코드 설명 보고서

항목	내용
앱 이름	EmotionBudgetApp
패키지	com.example.emotionbudgetapp
개발 방식	Kotlin + Jetpack Compose + Material3
구조	단일 Activity 기반 MVVM 형태: MainActivity -> Root Navigation -> 화면별 Composable -> ViewModel 상태
핵심 목적	수입/지출 기록에 감정 태그를 함께 저장해 소비 금액과 감정 패턴의 관계를 분석한다.
작성 기준	2026년 6월 5일, GitHub branch: codex/basic-ledger-features

1. 프로젝트 개요

EmotionBudgetApp은 일반 가계부에 감정 기록을 결합한 Android 앱이다. 사용자는 수입과 지출을 입력하면서 소비 당시의 감정, 카테고리, 날짜, 메모를 함께 저장한다. 앱은 이 데이터를 바탕으로 월별 지출 흐름, 날짜별 기록, 감정별 지출 금액과 소비 빈도, 스트레스 소비 변화, 감정 소비 위험도를 보여준다.

핵심 차별점은 단순히 얼마를 썼는지에서 끝나지 않고 어떤 감정일 때 소비가 반복되는지를 분석한다는 점이다. 특히 스트레스 소비 횟수와 비중을 진단 카드로 보여줘 사용자가 소비 습관을 돌아볼 수 있게 한다.

2. 기술 환경 및 라이브러리

사용 항목	사용 목적
Android Gradle Plugin 9.1.1	Android 앱 빌드, 패키징, SDK 설정을 담당한다.
Kotlin 2.2.10 / Kotlin Compose Plugin	Kotlin 문법과 Jetpack Compose 컴파일을 지원한다.
AndroidX Core KTX 1.18.0	Android API를 Kotlin 방식으로 간결하게 사용할 수 있게 한다.
Activity Compose 1.9.3	Activity에서 Compose UI를 실행하고 생명주기와 연결한다.
Jetpack Compose BOM 2024.10.00	Compose UI, Graphics, Tooling Preview, Material3 버전을 일관되게 맞춘다.
Material3	카드, 버튼, 텍스트 필드, 네비게이션 바 등 주요 UI 컴포넌트를 제공한다.
Lifecycle ViewModel Compose 2.8.6	ViewModel 상태를 Compose 화면에서 안전하게 관찰한다.
JUnit 4.13.2 / AndroidX Test / Espresso	ViewModel 단위 테스트와 Android 테스트 환경을 제공한다.

3. 프로젝트 구조

```
EmotionBudgetApp/
├── app/
│   ├── build.gradle.kts
│   └── src/main/java/com/example/emotionbudgetapp/
│       ├── MainActivity.kt
│       ├── data/Expense.kt
│       ├── viewModel/ExpenseViewModel.kt
│       └── ui/
│           ├── EmotionBudgetAppRoot.kt
│           ├── ExpenseScreen.kt
│           ├── ExpenseItem.kt
│           ├── LedgerReportScreen.kt
│           └── EmotionAnalysisScreen.kt
├── gradle/libs.versions.toml
└── docs/final-demo-flow.md
```

파일	역할
app/build.gradle.kts	앱 모듈의 SDK, Compose 사용 여부, 의존성, 테스트 설정을 정의한다.
gradle/libs.versions.toml	라이브러리와 플러그인 버전을 한곳에서 관리하는 Version Catalog이다.
MainActivity.kt	앱 시작 지점. ViewModel을 생성하고 루트 Compose 화면을 호출한다.
data/Expense.kt	수입/지출 타입과 기록 데이터 모델을 정의한다.
viewModel/ExpenseViewModel.kt	기록 목록 상태, 추가/수정/삭제, 합계 계산, 샘플 데이터 생성을 담당한다.
ui/EmotionBudgetAppRoot.kt	하단 네비게이션으로 기록, 통계, 감정 화면을 전환한다.
ui/ExpenseScreen.kt	기록 입력, 수정, 삭제, 검색, 기간/유형/카테고리/감정 필터를 제공한다.
ui/ExpenseItem.kt	목록에 표시되는 개별 수입/지출 카드 UI를 담당한다.
ui/LedgerReportScreen.kt	월별 이동, 일일/달력/주별/월별/요약 통계 화면을 담당한다.
ui/EmotionAnalysisScreen.kt	감정별 지출 금액/빈도 그래프, 스트레스 변화, 감정 소비 진단 카드를 담당한다.
ExpenseViewModelTest.kt	기록 추가, 수입/지출 합계 분리, 수정, 삭제, 카테고리 합계, 샘플 데이터 로직을 검증한다.

4. 주요 기능 요약

기능	설명
기본 가계부	금액, 수입/지출 유형, 날짜, 카테고리, 감정, 메모를 입력하고 목록에서 수정/삭제할 수 있다.
기록 찾기	검색어와 기간, 유형, 카테고리, 감정 조건을 조합해 원하는 기록을 찾고 결과 없음 상태를 보여준다.
상세 통계	월 이동 버튼과 일일/달력/주별/월별/요약 탭으로 기록을 여러 관점에서 확인한다.
감정 분석	감정별 총액, 횟수, 평균, 전월 대비 변화, 지출 비중을 계산한다.
소비 패턴 그래프	감정별 지출 금액과 소비 빈도를 막대 그래프로 시각화한다.
감정 소비 진단	스트레스 소비 횟수, 증가 여부, 비중, 대표 감정을 점수화해 위험도를 제시한다.
시연 데이터	샘플 데이터를 불러와 제출 영상에서 통계와 감정 분석 화면을 즉시 보여줄 수 있다.

5. 앱 실행 흐름

```

MainActivity
-> EmotionBudgetAppRoot
  -> ExpenseScreen: 기록 입력/수정/삭제/검색
  -> LedgerReportScreen: 월별/일별/주별/요약 통계
  -> EmotionAnalysisScreen: 감정별 그래프와 진단

ExpenseViewModel
-> StateFlow<List<Expense>>
-> 모든 화면이 같은 기록 목록을 관찰하고 재계산
  
```

이 구조는 화면 이동과 데이터 상태를 분리한다. Activity는 앱 진입점만 담당하고, Root 화면은 네비게이션만 담당하며, ViewModel은 기록 목록과 기본 계산을 담당한다. 각 화면은 자신에게 필요한 통계만 계산해 UI로 표현한다.

6. 핵심 코드 설명

1. 데이터 모델: 기록의 기준 형식

파일: app/src/main/java/com/example/emotionbudgetapp/data/Expense.kt

```

enum class TransactionType(val label: String) {
    INCOME("수입"),
    EXPENSE("지출")
}

data class Expense(
    val id: Int,
    val amount: Int,
    val category: String,
    val emotion: String,
    val memo: String,
    val dateMillis: Long,
    val type: TransactionType = TransactionType.EXPENSE
)
  
```

- TransactionType으로 수입과 지출을 분리해 합계가 섞이지 않게 했다.
- Expense는 화면, 검색, 통계, 감정 분석이 공통으로 사용하는 중심 데이터 모델이다.
- dateMillis를 저장해 오늘, 이번 주, 이번 달, 월별 이동 같은 기간 필터의 기준으로 사용한다.
- emotion 필드는 이 앱의 차별점으로, 감정별 소비 패턴 분석의 입력값이 된다.

2. ViewModel: 상태 관리와 CRUD

파일: app/src/main/java/com/example/emotionbudgetapp/viewmodel/ExpenseViewModel.kt

```

private val _expenses = MutableStateFlow<List<Expense>>(emptyList())
val expenses: StateFlow<List<Expense>> = _expenses
private var nextId = 1

fun addExpense(...) {
    val newExpense = Expense(id = nextId++, ...)
    _expenses.value = _expenses.value + newExpense
}

fun updateExpense(...) {
    _expenses.value = _expenses.value.map { expense ->
        if (expense.id == id) expense.copy(...) else expense
    }
}

fun deleteExpense(expense: Expense) {
    _expenses.value = _expenses.value.filter { it.id != expense.id }
}

fun getBalance(): Int {
    return getIncomeTotal() - getExpenseTotal()
}
  
```

- MutableStateFlow에 기록 목록을 저장하고 Compose 화면은 collectAsState()로 이를 관찰한다.
- 리스트를 직접 수정하지 않고 새 리스트로 교체해 화면이 안정적으로 다시 그려지도록 했다.
- id를 기준으로 수정/삭제 대상을 찾기 때문에 같은 카테고리나 금액의 기록이 있어도 안전하게 처리된다.
- getIncomeTotal(), getExpenseTotal(), getBalance()로 수입/지출/잔액 계산을 분리했다.

3. 화면 이동: 하단 탭 기반 앱 구조

파일: app/src/main/java/com/example/emotionbudgetapp/ui/EmotionBudgetAppRoot.kt

```
private enum class AppDestination(val label: String) {
    Ledger("기록"),
    Report("통계"),
    Emotion("감정")
}

Scaffold(bottomBar = { NavigationBar { ... } }) { innerPadding ->
    when (currentDestination) {
        AppDestination.Ledger -> ExpenseScreen(viewModel = viewModel)
        AppDestination.Report -> LedgerReportScreen(expenses = expenses, ...)
        AppDestination.Emotion -> EmotionAnalysisScreen(expenses = expenses, ...)
    }
}
```

- 한 화면에 모든 기능을 넣지 않고 기록, 통계, 감정 분석 화면으로 역할을 나눴다.
- 하단 네비게이션을 사용해 모바일 앱에서 익숙한 방식으로 화면을 이동할 수 있다.
- Root 화면에서 ViewModel 상태를 읽어 통계/감정 화면에 전달하므로 데이터 흐름이 단순하다.

4. 기록 찾기: 검색어와 필터 조합

파일: app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt

```
val filteredExpenses = expenses.filter { expense ->
    val query = appliedSearchText.trim()
    val matchesQuery = query.isBlank() ||
        expense.type.label.contains(query, ignoreCase = true) ||
        expense.category.contains(query, ignoreCase = true) ||
        expense.emotion.contains(query, ignoreCase = true) ||
        expense.memo.contains(query, ignoreCase = true) ||
        formatWon(expense.amount).contains(query)

    val matchesPeriod = matchesPeriodFilter(...)
    val matchesType = typeFilter == "전체" || expense.type.label == typeFilter
    val matchesCategory = categoryFilter == "전체" || expense.category == categoryFilter
    val matchesEmotion = emotionFilter == "전체" ||
        (expense.type == TransactionType.EXPENSE && expense.emotion == emotionFilter)

    matchesQuery && matchesPeriod && matchesType && matchesCategory && matchesEmotion
}
```

- 검색어는 유형, 카테고리, 감정, 메모, 날짜, 금액을 모두 대상으로 삼는다.
- 기간, 유형, 카테고리, 감정 조건을 동시에 적용해 실제 가계부 앱처럼 원하는 기록을 좁혀 찾는다.
- 필터 결과 개수와 결과 없음 안내를 UI에 표시해 검색 버튼을 눌렀을 때 반응이 명확하다.

5. 상세 통계: 월별 데이터 계산과 탭 전환

파일: app/src/main/java/com/example/emotionbudgetapp/ui/LedgerReportScreen.kt

```
val monthStart = startOfMonth(visibleMonthMillis)
val monthEnd = addMonths(monthStart, 1)
val monthRecords = expenses
    .filter { it.dateMillis >= monthStart && it.dateMillis < monthEnd }
    .sortedWith(compareByDescending<Expense> { it.dateMillis }).thenByDescending { it.id }

val monthIncomeTotal = monthRecords
    .filter { it.type == TransactionType.INCOME }
    .sumOf { it.amount }
val monthExpenseTotal = monthRecords
    .filter { it.type == TransactionType.EXPENSE }
    .sumOf { it.amount }

when (selectedTab) {
    ReportTab.Daily -> buildDailyGroups(monthRecords)
    ReportTab.Calendar -> CalendarMonthView(monthStart, monthRecords)
    ReportTab.Weekly -> WeeklyReportView(monthStart, monthRecords)
    ReportTab.Monthly -> MonthlyReportView(monthRecords)
    ReportTab.Summary -> SummaryReportView(monthStart, monthRecords)
}
```

- 현재 선택된 달의 시작일과 다음 달 시작일을 기준으로 월 기록을 정확히 자른다.
- 수입과 지출 합계를 따로 계산해 월별 총수입, 총지출, 잔액을 표시한다.
- 같은 데이터를 일일, 달력, 주별, 월별, 요약 탭으로 재구성해 기록이 많아져도 보기 쉽다.

6. 감정별 분석: 금액, 빈도, 전월 대비 변화

파일: app/src/main/java/com/example/emotionbudgetapp/ui/EmotionAnalysisScreen.kt

```
private fun buildEmotionStats(
    monthExpenses: List<Expense>,
    previousMonthExpenses: List<Expense>
): List<EmotionStat> {
    val defaultEmotions = listOf("기쁨", "슬픔", "스트레스", "외로움", "평온", "분노")
    val allEmotions = (defaultEmotions + monthExpenses.map { it.emotion } +
        previousMonthExpenses.map { it.emotion }).distinct()
    val totalAmount = monthExpenses.sumOf { it.amount }.takeIf { it > 0 } ?: 1

    return allEmotions.map { emotion ->
        val current = monthExpenses.filter { it.emotion == emotion }
        val previous = previousMonthExpenses.filter { it.emotion == emotion }
        val currentTotal = current.sumOf { it.amount }
        EmotionStat(
            emotion = emotion,
            totalAmount = currentTotal,
            count = current.size,
            averageAmount = if (current.isEmpty()) 0 else currentTotal / current.size,
            previousCount = previous.size,
            countChange = current.size - previous.size,
            share = currentTotal.toFloat() / totalAmount.toFloat()
        )
    }
```

```
}
}
```

- 이번 달 지출과 지난달 지출을 비교해 감정별 횟수 변화까지 계산한다.
- 총액, 횟수, 평균, 비중을 함께 저장해 카드 UI와 그래프 UI에서 재사용한다.
- 기본 감정 목록을 유지하면서 사용자가 새로운 감정을 추가해도 분석 대상에 포함된다.

7. 감정 소비 진단: 차별화 기능

파일: app/src/main/java/com/example/emotionbudgetapp/ui/EmotionAnalysisScreen.kt

```
val stressCountScore = (stressCount * 12).coerceAtMost(30)
val stressIncreaseScore = if (stressChange > 0) 20 else 0
val stressShareScore = if (stressShare >= 0.3f) 25 else 0
val primaryStressScore = if (primaryEmotion == "스트레스") 15 else 0
val riskScore = (
    20 + stressCountScore + stressIncreaseScore +
    stressShareScore + primaryStressScore
).coerceIn(0, 100)

val riskLabel = when {
    riskScore >= 70 -> "높음"
    riskScore >= 40 -> "보통"
    else -> "낮음"
}
```

- 단순 통계가 아니라 스트레스 소비의 반복성, 증가 여부, 지출 비중을 종합해 위험도를 계산한다.
- 결과를 높음/보통/낮음으로 구분해 사용자가 바로 이해할 수 있게 했다.
- 진단 문구와 추천 행동을 함께 보여줘 앱의 창의성 평가 항목에 어필할 수 있다.

8. 단위 테스트: 핵심 계산 검증

파일: app/src/test/java/com/example/emotionbudgetapp/ExpenseViewModelTest.kt

```
@Test
fun addExpense_separatesIncomeAndExpenseTotals() {
    val viewModel = ExpenseViewModel()
    viewModel.addExpense(3000000, "급여", "평온", "월급", 1000L, TransactionType.INCOME)
    viewModel.addExpense(12000, "식비", "기쁨", "점심", 2000L, TransactionType.EXPENSE)

    assertEquals(3000000, viewModel.getIncomeTotal())
    assertEquals(12000, viewModel.getExpenseTotal())
    assertEquals(2988000, viewModel.getBalance())
}
```

- 수입과 지출 합계가 섞이지 않는지 자동 테스트로 확인한다.
- 추가, 수정, 삭제, 카테고리 합계, 샘플 데이터도 테스트해 기본 기능의 안정성을 높였다.
- 제출 전 gradlew.bat test와 gradlew.bat assembleDebug로 테스트와 빌드를 검증했다.

7. 검증 내용

검증 항목	결과
단위 테스트	gradlew.bat test 통과
디버거 빌드	gradlew.bat assembleDebug 통과
테스트 범위	기록 추가, 수입/지출 합계 분리, 수정, 삭제, 카테고리 합계, 샘플 데이터 검증
시연 준비	docs/final-demo-flow.md에 3분 이내 영상 촬영 흐름 정리

8. 현재 한계와 개선 방향

항목	설명
데이터 저장	현재는 MutableStateFlow 기반 메모리 저장 구조이다. 제출 영상과 기능 검증에는 충분하지만, 앱을 종료해도 기록을 유지하려면 Room DB 또는 DataStore 확장이 필요하다.
사용자 설정	카테고리와 감정 목록은 화면 내부 기본값을 중심으로 동작한다. 장기 사용 앱으로 발전시키려면 사용자 정의 카테고리/감정 관리 화면을 추가할 수 있다.
차트 확장	현재 그래프는 Compose 레이아웃으로 직접 만든 막대 그래프이다. 더 복잡한 통계가 필요하면 차트 라이브러리나 월별 비교 그래프를 추가할 수 있다.

9. 제출 관점에서의 핵심 어필 포인트

- 기획 및 요구 분석: 감정 기반 소비 관리라는 목적과 타깃이 명확하고, 수입/지출 기록부터 분석까지 기능 명세가 연결되어 있다.
- UI/UX 디자인: 기록, 통계, 감정 화면을 하단 탭으로 분리해 복잡도를 낮췄고, 검색 결과와 빈 상태 문구를 명확히 제공한다.
- 기능 구현 및 완성도: CRUD, 수입/지출 분리, 월별 통계, 검색/필터, 감정 분석, 그래프, 진단 카드, 테스트가 구현되어 있다.
- 창의성 및 차별성: 감정별 지출 분석과 스트레스 소비 진단은 일반 가계부와 구분되는 대표 기능이다.